



UNIVERSITY OF GENEVA

Faculty of Science

Exercise #9

Introduction to Computational Finance
14X030

Michel Jean Joseph Donnet



Contents

1 Option pricing: Black-Scholes versus Binomial Tree	3
2 Implied Volatility from Binomial Prices	6
2.1 Strikes & Tree Depths	6
2.2 Compute Tree Prices	6
2.3 Implied Volatility via Bisection Method	6
2.4 Volatility Smile Plot	7



1 Option pricing: Black-Scholes versus Binomial Tree

Let an asset S be valued at $t = 0$ at $S_0 = 100$. We consider a European option to buy this asset (call) with maturity $T = 1$ (in years) and strike price $K = 120$. The goal of this exercise is to compare two methods to price this option, Black-Scholes and Binomial Tree. We assume a constant volatility $\sigma = 20\%$ over the lifespan of the call, and a risk-free rate $r = 5\%$.

1. Implement the Black-Scholes formula to determine the value of this call at $t = 0$.

```
import numpy as np
from scipy.stats import norm

def black_scholes_call(S0: float, K: float, T: float, r: float, sigma:
float) -> float:
    """
    Price a European call option using the Black-Scholes formula.

    Parameters:
    S0: float
        Initial stock price
    K: float
        Strike price
    T: float
        Time to maturity (in years)
    r: float
        Risk-free interest rate
    sigma: float
        Volatility of the underlying asset

    Returns:
    float: Estimated call option price
    """
    d1 = (np.log(S0 / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    call_price = S0 * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
    return call_price

S0 = 100
K = 120
T = 1
r = 0.05
sigma = 0.20
call_price_bs = black_scholes_call(S0, K, T, r, sigma)
print(f"Black-Scholes Call Price: {call_price_bs:.2f}")
```

Black-Scholes Call Price: 3.25



2. Implement a binomial tree to determine the initial value of the call. Your implementation should take the depth of the tree as an argument.

```
def binomial_call(
    S0: float, K: float, T: float, r: float, sigma: float, N: int
) -> float:
    """
    Price a European call option using a binomial tree.

    Parameters:
    S0: float
        Initial stock price
    K: float
        Strike price
    T: float
        Time to maturity (in years)
    r: float
        Risk-free interest rate
    sigma: float
        Volatility of the underlying asset
    N: int
        Number of steps in the binomial tree

    Returns:
    float: Estimated call option price
    """
    dt = T / N
    u = np.exp(sigma * np.sqrt(dt))
    d = np.exp(-sigma * np.sqrt(dt))
    p = (np.exp(r * dt) - d) / (u - d)

    # Initialize asset prices at maturity
    asset_prices = np.zeros(N + 1)
    for i in range(N + 1):
        asset_prices[i] = S0 * (u**i) * (d ** (N - i))

    # Initialize option values at maturity
    # If negative, the option is not exercised, hence max(0, S-K)
    option_values = np.maximum(0, asset_prices - K)

    # Step back through the tree
    for j in range(N - 1, -1, -1):
        option_values[: j + 1] = np.exp(-r * dt) * (
            p * option_values[1 : j + 2] + (1 - p) * option_values[: j + 1]
        )

    return option_values[0]

N = 100 # Tree depth
call_price_tree = binomial_call(S0, K, T, r, sigma, N)
print(f"Binomial Tree Call Price (N={N}): {call_price_tree:.2f}")
```



Binomial Tree Call Price (N=100): 3.26

The result obtained from the binomial tree is close to the Black-Scholes price: 3.26 is very close to 3.25, which is the price obtained from the Black-Scholes formula.

- On the same graph, plot the evolution of the estimated value of the call option as a function of the binomial tree depth, as well as the value derived with Black-Scholes. What do you observe? How deep should be the tree in order to get a reasonable approximation of the Black-Scholes value?

```
import matplotlib.pyplot as plt

depths = [10, 50, 100, 200, 500]
prices = [binomial_call(S0, K, T, r, sigma, N) for N in depths]

plt.figure(figsize=(10, 6))
plt.plot(depths, prices, marker="o", label="Binomial Tree Price")
plt.axhline(y=call_price_bs, color="r", linestyle="--", label="Black-Scholes Price")
plt.xscale("log")
plt.xlabel("Binomial Tree Depth (N)")
plt.ylabel("Call Option Price")
plt.title("Call Option Price vs. Binomial Tree Depth")
plt.legend()
plt.grid()
plt.show()
```

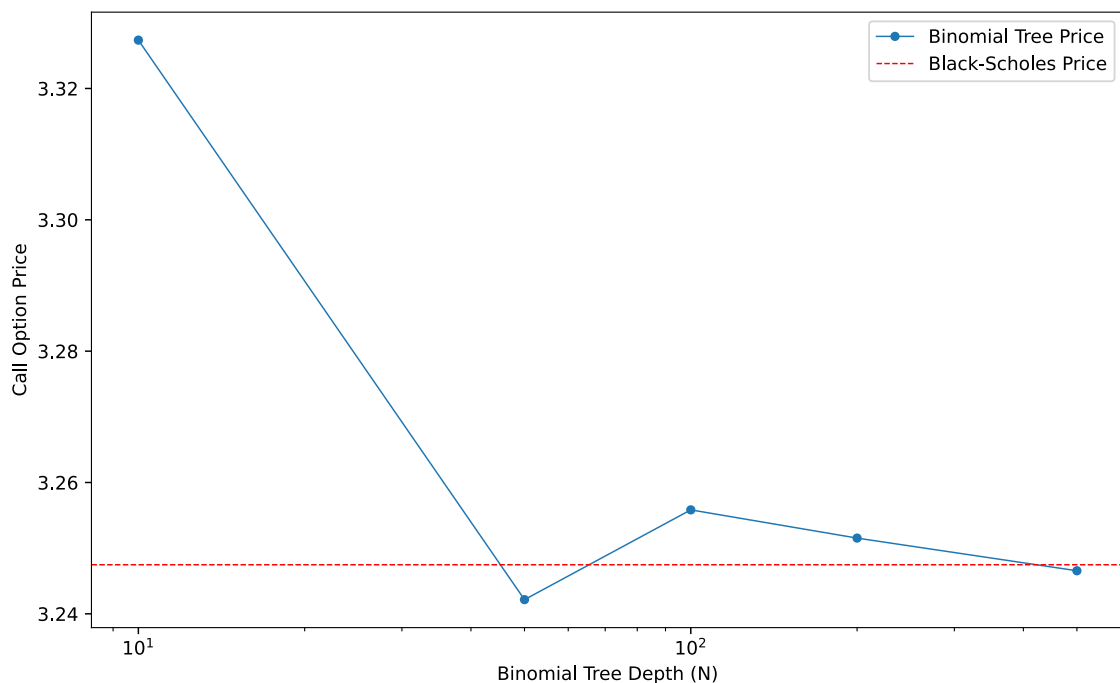


Figure 1: Call Option Price vs. Binomial Tree Depth

We can observe that with a few depths (e.g., $N=10$), the binomial tree price is quite far from the Black-Scholes price. As we increase the depth, the binomial tree price converges towards the Black-Scholes price.



To get a reasonable approximation, we might need a depth of around $N=100$ or more since we are more close to the Black-Scholes price at that point.

So the approximation accuracy can be controlled by the depth of the tree, allowing us to balance between computational cost and precision.

2 Implied Volatility from Binomial Prices

Using your binomial-tree pricer, compute the implied Black-Scholes volatility for different strikes and tree depths. Plot the resulting volatility “smile” and discuss convergence as the tree deepens.

2.1 Strikes & Tree Depths

- Fix $S_0 = 100$, $T = 1$, $r = 0.05$.
- Consider strikes $K \in \{80, 90, 100, 110, 120\}$.
- Use three binomial-tree depths: $N \in \{20, 100, 500\}$.

```
S0 = 100
T = 1
r = 0.05
strikes = [80, 90, 100, 110, 120]
tree_depths = [20, 100, 500]
```

2.2 Compute Tree Prices

For each (K, N) , compute the call price

$$C_{\text{tree}} = \text{binomial_call}(S_0, K, T, r, \sigma_{\text{true}}, N), \quad \sigma_{\text{true}} = 0.20.$$

```
sigma_true = 0.20
tree_prices = {
    (K, N): binomial_call(S0, K, T, r, sigma_true, N)
    for K in strikes
    for N in tree_depths
}
```

2.3 Implied Volatility via Bisection Method

- Implement the Black-Scholes call price

$$C_{\text{BS}}(S_0, K, T, r, \sigma) = S_0 N(d_1) - K e^{-rT} N(d_2),$$

with



$$d_{1,2} = \frac{\ln(S_0/K) + (r \pm \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}.$$

- For each tree price C_{tree} , solve for σ_{imp} satisfying

$$C_{\text{BS}}(S_0, K, T, r, \sigma_{\text{imp}}) = C_{\text{tree}}$$

by using a bisection¹ over $\sigma \in [10^{-4}, 2.0]$ with tolerance 10^{-6} .

```
from scipy.optimize import bisect

implied_vols = {}
for (K, N), C_tree in tree_prices.items():

    def f(sigma_imp):
        return black_scholes_call(S0, K, T, r, sigma_imp) - C_tree

    sigma_imp = bisect(f, 1e-4, 2.0, xtol=1e-6)
    implied_vols[(K, N)] = sigma_imp
```

2.4 Volatility Smile Plot

- On one chart, plot $\sigma_{\text{imp}(K)}$ vs. K for each tree depth N , then add a horizontal line at $\sigma_{\text{true}} = 0.20$ as well.

```
plt.figure(figsize=(10, 6))
for N in tree_depths:
    sigmas = [implied_vols[(K, N)] for K in strikes]
    plt.plot(strikes, sigmas, marker="o", label=f"Tree Depth N={N}")
plt.axhline(y=sigma_true, color="r", linestyle="--", label="True Volatility (20%)")
plt.xlabel("Strike Price (K)")
plt.ylabel("Implied Volatility")
plt.title("Implied Volatility Smile from Binomial Tree Prices")
plt.legend()
plt.grid()
plt.show()
```

¹ Instead of coding the bisection loop yourself, you can use Python's `scipy.optimize.bisect`. Define $f(\sigma) = C_{\text{BS}}(S_0, K, T, r, \sigma) - C_{\text{tree}}$. Call `bisect(f, 1e-4, 2.0, xtol=1e-6)` to find σ_{imp} . This will be more concise and handles convergence for you.

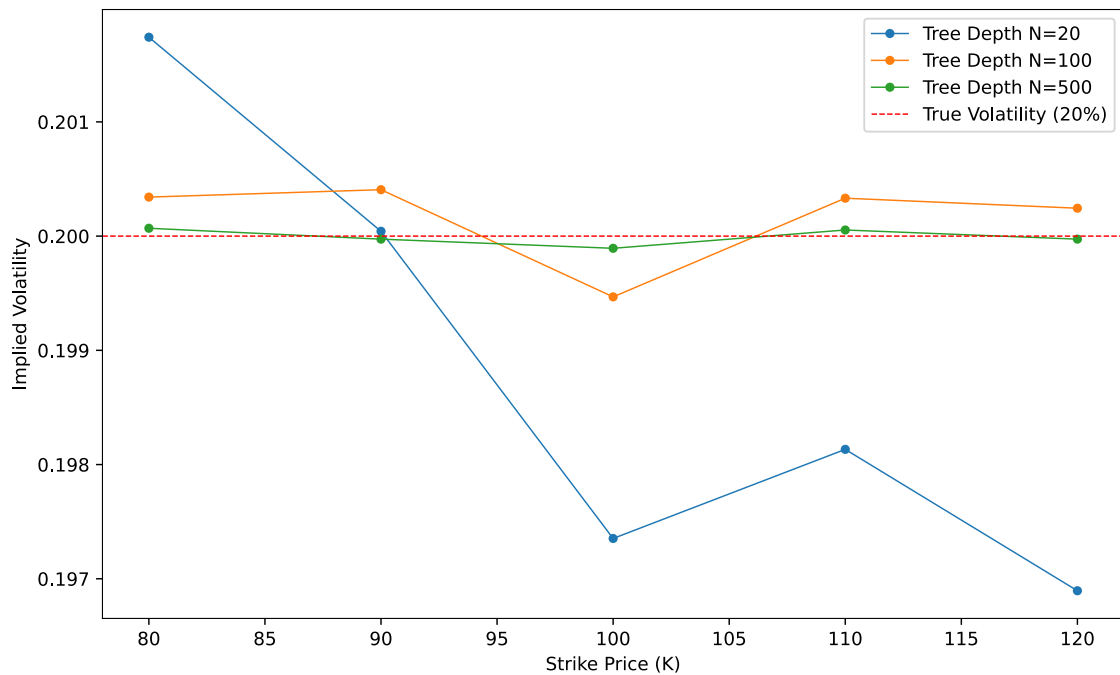


Figure 2: Implied Volatility Smile from Binomial Tree Prices

- Comment on how the smile flattens as N increases.

When the depth of the tree increases, the implied volatility smiles tend to be more flat and closer to the true volatility computed by the Black-Scholes model.

Indeed, the Black-Scholes model is a continuous-time model, while the binomial tree is a discrete approximation. As we increase the number of steps in the tree, we get a better approximation of the continuous process, which leads to implied volatilities that are more consistent across different strikes and closer to the true volatility, as we can see on the plot where for instance at $N=500$, the implied volatilities very close to the Black-Scholes volatility across all strikes.